Review of literature on Performance and Architectural Trade-offs in Kubernetes Container Network Interfaces

Name_RollNo
Course Name
IIIT-Delhi

March 16, 2026

1 Introduction

Kubernetes (K8s) has revolutionized and emerged as the industry standard for orchestration of containerized application, providing scaling from massive cloud-native architectures to highly constrained edge computing environments. At the core to this ecosystem is the Container Network Interface (CNI), which manage the network communication flows between distributed pods and nodes. The selection of a CNI plugin fundamentally impacts the overall latency, bandwidth efficiency, and computational overhead of the entire cluster.

As applications evolved to require higher throughput for data-intensive workloads or lower latency for real-time edge processing, the architectural paradigm of CNIs has also shifted. Traditional overlay networks relying on Linux kernel frameworks like iptables are now being evaluated against pure routing underlays and emerging extended Berkeley Packet Filter (eBPF) technologies. This literature reviews the performance of various CNI plugins such as Flannel, Calico, Antrea and Cillium by analyzing how their design choices are influenced under system-tuning profiles, their resource footprints on lightweight edge distributions, and the architectural trade-offs between traditional routing and eBPF processing.

2 Performance Overhead and System Tuning

When container networking is analysed against bare-metal(local system like windows, linux, etc.), non-CNI physical networks, a substantial degradation in performance is routinely observed regardless of the chosen plugin. Evaluations that measured TCP and UDP bandwidth efficiency indicates the major CNI plugins struggle to achieve full link saturation, particularly as packet sizes increase beyond standard MTU parameters [1]. For UDP traffic utilizing a 9000 MTU size, the bandwidth performance drop is almost universally around 40-50% compared to physical network evaluations [1].

However, applying specific system-tuning profiles directly impacts the efficiency of these plugins. For instance, introducing kernel-level optimizations such as increasing receive/send buffer sizes and enabling TCP window scaling-significantly improves both TCP and UDP throughput for the Calico plugin [1]. In these optimized scenarios, Calico can outperform other CNIs by approximately 2 Gbit/s [1].

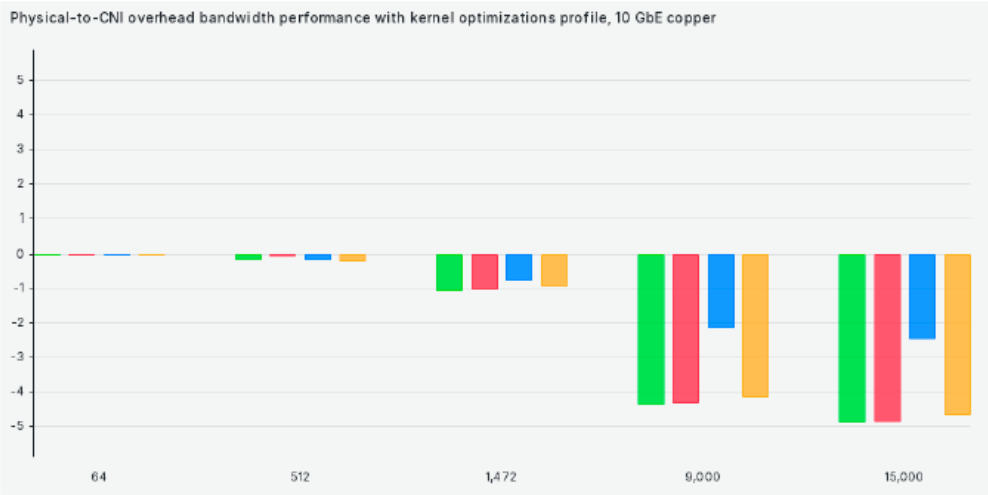


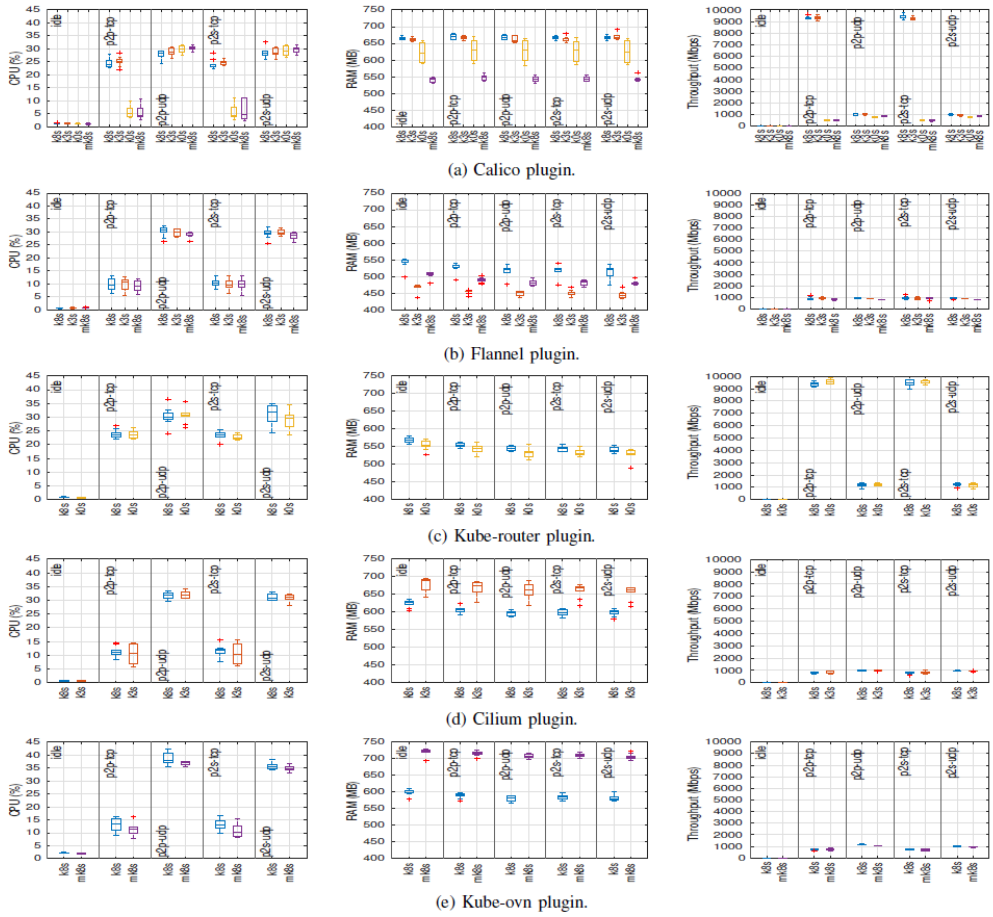
Figure 1: TCP Bandwidth Efficiency Comparison across CNI Plugins under Kernel Optimization Profiles.

In terms of transmission delays, the Flannel plugin repeatedly exhibits the lowest latency overhead across a variety of tuned system profiles when compared to its peers [1]. This is considered counter-intuitive, as Flannel relies on VXLAN encapsulation, a process that typically introduces latency overhead [1]. Despite this, Flannel manages to maintain a clear latency advantage in both TCP and UDP evaluations, often by a margin of over 20% [1].

3 CNI Footprints in Resource-Constrained Edge Environments

Extending container orchestration to the edge requires lightweight distributions like K3s, K0s, and MicroK8s. These distributions are designed to minimize their operating system dependencies to accommodate devices with stringent CPU and RAM constraints [2]. However, deploying networking plugins within these trimmed environments yields highly varied resource utilization patterns.

Research demonstrates that the CPU utilization required by CNI plugins remains largely static, regardless of whether they are executed on a robust vanilla Kubernetes cluster or a lightweight edge distribution [2]. An exception occurs when plugins shift from underlay to overlay mechanisms within specific distributions, which can slightly alter computational demand [2].



(a) Calico plugin.

(b) Flannel plugin.

(c) Kube-router plugin.

(d) Cilium plugin.

(e) Kube-ovn plugin.

Figure 2: CPU and RAM Utilization Variations of CNI Plugins across Vanilla and Lightweight Edge K8s Distributions.

Conversely, memory footprint behaviors vary drastically depending on the plugin's underlying architecture. While standard plugins like Flannel and Calico successfully minimize their RAM consumption on edge distributions, more complex solutions suffer in these environments. Specifically, Flannel reduces its RAM footprint by 28% in K3s and 15% in MicroK8s [2]. In contrast, the eBPF-based Cilium and the SDN-based Kube-ovn actually experience increased RAM consumption (14% and 16% respectively) when deployed on these exact same lightweight distributions [2].

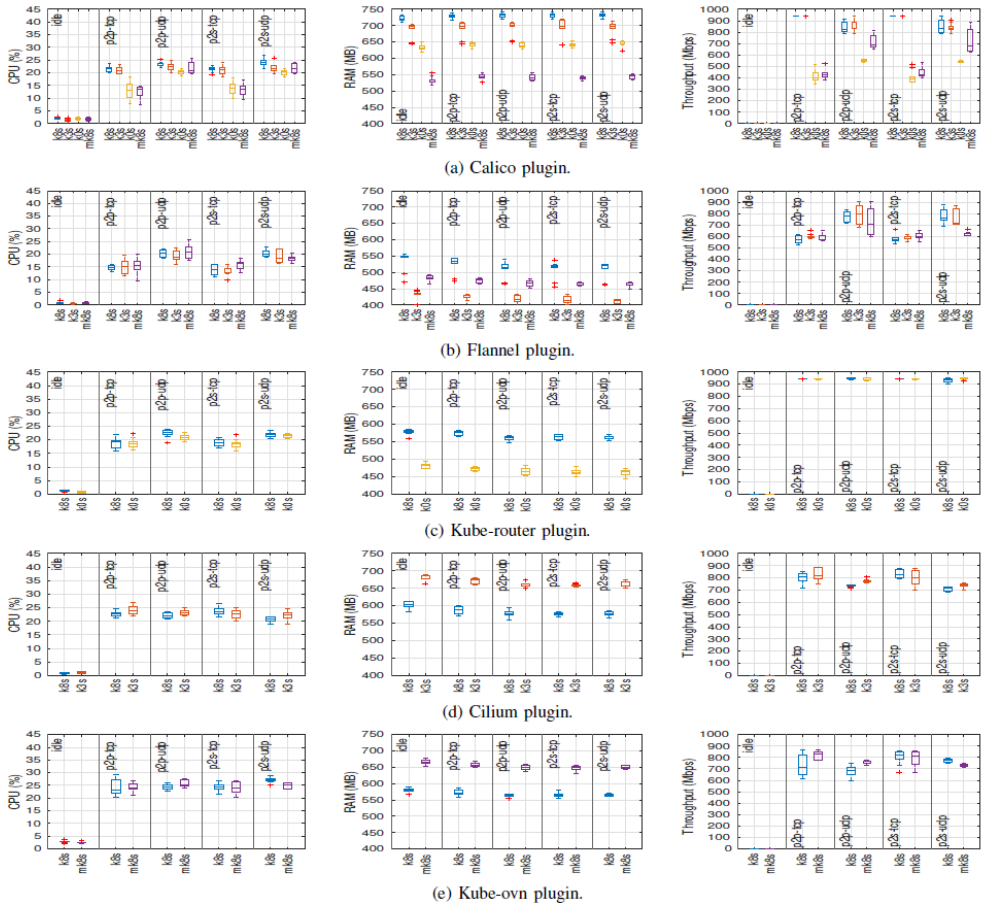


Figure 3: Throughput Performance Limitations in Inter-host vs. Intra-host Edge Communication Scenarios.

Furthermore, the reduction of a plugin's resource footprint does not necessarily translate to improved throughput [2]. In edge deployments, pure IP-based routing plugins consistently provide superior throughput compared to overlay-based solutions, though no plugin entirely avoids the performance degradation inherent to highly constrained physical hardware [2].

4 The Architectural Shift to eBPF-Based Data Planes

Modern Kubernetes networking is currently undergoing a paradigm shift from traditional, iptables-based routing frameworks to eBPF-enabled processing. By executing user-defined programs directly within the kernel, eBPF dynamically extends functionality without requiring system reboots or source code modifications [3]. In the context of Kubernetes, plugins like Cilium leverage eBPF to bypass the standard kube-proxy component, managing routing and packet filtering directly and efficiently [3].

Under idle or low-load cluster conditions, traditional iptables setups actually demonstrate superior performance metrics. Baseline assessments show that an unloaded traditional setup achieves higher average throughput (approximately 7850 Mbps) compared to an eBPF-enabled configuration (7071 Mbps) [3]. The traditional setup also maintains a slight advantage in response times, exhibiting lower packet latency [3].

Table 1: Test results for an unloaded cluster without eBPF (iptables)

Test number	Throughput (Mbps)	Latency (μs)	CPU Usage (%)	RAM Usage (MB)
1	7660.57	488.859	19.21	1256.90
2	7757.71	486.943	19.30	1259.47
3	7867.42	486.655	19.24	1261.90
4	7864.38	481.298	19.24	1264.80
5	8104.38	485.396	19.41	1258.00
Avg	**7850.89**	**485.830**	**19.28**	**1260.21**

Table 2: Test results for an unloaded cluster with eBPF

Test number	Throughput (Mbps)	Latency (μs)	CPU Usage (%)	RAM Usage (MB)
1	6840.76	504.884	19.28	1248.38
2	6901.81	500.168	19.58	1248.72
3	7176.38	501.531	19.14	1251.90
4	7186.28	502.710	19.07	1257.14
5	7252.80	505.814	19.20	1263.90
Avg	**7071.61**	**503.021**	**19.25**	**1254.01**

However, when a cluster is subjected to heavy, production-simulated application loads, the architectural advantages of eBPF become distinctly clear. While the traditional routing infrastructure still maintains a slight edge in raw data transfer speeds (6786 Mbps versus 6459 Mbps), the eBPF architecture becomes highly efficient regarding system resources [3].

Table 3: Test results for a loaded cluster without eBPF (iptables)

Test number	Throughput (Mbps)	Latency (μs)	CPU Usage (%)	RAM Usage (MB)
1	6688.00	546.410	47.62	1991.70
2	6797.09	514.680	46.20	1996.63
3	6801.90	548.652	45.10	1971.85
4	6986.90	544.042	45.47	1980.22
5	6660.19	552.555	44.75	1970.47
Avg	**6786.82**	**541.268**	**45.83**	**1982.17**

Table 4: Test results for a loaded cluster with eBPF

Test number	Throughput (Mbps)	Latency (μs)	CPU Usage (%)	RAM Usage (MB)
1	6412.19	535.215	41.22	1768.00
2	6344.00	488.831	40.19	1772.63
3	6556.36	511.912	41.21	1775.63
4	6631.27	505.682	40.81	1761.90
5	6352.72	516.650	41.25	1771.54
Avg	**6459.31**	**511.658**	**40.94**	**1769.94**

Under heavy load, eBPF-enabled clusters display significantly reduced CPU consumption, averaging 40.94% compared to the traditional cluster's 45.83% [3]. Memory efficiency also heavily favors eBPF

under load, with RAM consumption averaging nearly 200 MB less than the iptables equivalent [3]. Furthermore, the eBPF configuration provides mild improvements in responsiveness, achieving lower average latency during high-demand scenarios [3].

5 Conclusion and Open Questions

The current body of literature confirms that a universally optimal Kubernetes networking solution does not exist; rather, CNI selection must be strictly aligned with the specific operational demands of the deployment environment. For lightweight, resource-constrained edge deployments where minimizing RAM overhead and latency is critical, Flannel provides the most efficient architecture, albeit at the cost of advanced security features. For environments demanding maximum throughput, Calico excels, particularly when paired with intentional kernel-level optimizations. Finally, as clusters scale and encounter heavy application loads, transitioning to eBPF technologies like Cilium is highly recommended to maximize CPU and memory efficiency while maintaining competitive latency.

Moving forward, several open questions remain. As bandwidth requirements push toward 100 Gbit/s and beyond, future research must investigate hardware offloading techniques such as leveraging FPGAs or Data Processing Units (DPUs) to mitigate the inherent encapsulation and routing overheads generated by these CNI plugins. Additionally, standardizing the measurement of the "observability tax" introduced by deep packet inspection in eBPF plugins requires further empirical quantification.

References

[1] V. Dakić, J. Redžepagić, M. Bašić, and L. Žgrablić, “Performance and Latency Efficiency Evaluation of Kubernetes Container Network Interfaces for Built-In and Custom Tuned Profiles,” *Electronics*, vol. 13, no. 19, p. 3972, Oct. 2024.

[2] G. Koukis, S. Skaperas, I. A. Kapetanidou, L. Mamatas, and V. Tsaoussidis, “Performance Evaluation of Kubernetes Networking Approaches across Constraint Edge Environments,” *arXiv preprint arXiv:2401.07674*, Jan. 2024.

[3] K. Miziński and S. Przyłucki, “The impact of using eBPF technology on the performance of networking solutions in a Kubernetes cluster,” *Journal of Computer Sciences Institute (JCSI)*, vol. 35, pp. 150-158, 2025.